Contender 20: Third-Order Autonomous Hyperreflection in BusyLean

A constructive large-number definition, domination argument, and Lean integration package

Research proposal for the BusyLean project

13 October 2026

Abstract

BusyLean formalizes a "name the biggest number" contest in Lean 4: each entry must be a closed computable natural number, and each improvement must be accompanied by a formal strict inequality. At the audited repository head, Contender 19 combines two powerful components. Its object language is a strongly normalizing ladder of universes indexed by Schütte-style large-Veblen terms, and its amplifier is a natural hierarchy of *transformer lifts* constructed by diagonal hyperreflection.

This article proposes Contender 20, which raises the amplifier by one explicit type level. A new *lift transformer* maps transformer lifts to transformer lifts. We define coordinatewise splicing of whole lifts, a promotion operation that internalizes one successor step of the accepted diagonal hierarchy, a well-founded tree recursion over lift transformers, an autonomous third-order closure, and a finite meta-rank iteration of that closure. Both the schedule and the two ranks are controlled by the complete Contender 19 family at the current input. The resulting lift acts on large-Veblen reflection to produce a countable family; one final large-Veblen reflection then sees that whole family. The proposed closed number is

$$\mathsf{C}_{20} = F_{20}(F_{20}(42)).$$

The comparison architecture is intentionally stronger than the contest's minimum. The central invariant `GoodLift` says that a transformer lift preserves progressive transformers and is strictly inflationary on them. It is preserved by splicing, promotion, tree iteration, autonomous closure, and every finite meta-rank. The main architectural theorem states that the final third-order lift strictly exceeds its entire initial current-style `diagonalLift` at every visible coordinate. Pointwise domination of the Contender 19 family and a final input-growth argument then yield the required statement

$$\mathsf{C}_{19} < \mathsf{C}_{20}.$$

A second theorem bounds every enumerated large-Veblen-ladder program run relative to the final packed third-order family.

All definitions are structural recursions on natural numbers or well-founded W-trees and are computable in principle. This package includes Lean sources, a focused axiom/dependency audit, a clean-build timing harness, a guarded installer, a unified patch, source-provenance data, diagrams, and a compiled PDF. The packaging environment did not contain Lean or Lake. Accordingly, the article distinguishes source-level evidence from pending Lean 4.34.0 kernel replay and makes no unverified axiom or timing claim.

Contents

1 Problem, baseline, and research standard **4**

1.1 Contest constraints 4

1.2 Meaning of "significantly exceeds" 4

2 The accepted machinery through Contender 19 **5**
2.1 Families and transformers . . . 5
2.2 Autonomous lifting . . . 5
2.3 Diagonal hyperreflection . . . 6
2.4 The large-Veblen object language . . . 6

3 Design alternatives and the chosen increase **6**

4 Third-order autonomous hyperreflection **7**
4.1 Splicing whole lifts . . . 8
4.2 Promotion internalizes one diagonal successor . . . 8
4.3 Tree recursion over lift transformers . . . 8
4.4 Autonomous closure and finite meta-rank . . . 9

5 The invariant that makes comparison possible **9**
5.1 Two accepted properties . . . 9
5.2 Splicing preserves the invariant . . . 10
5.3 Promotion is good and lift-strict . . . 10
5.4 The node argument one level higher . . . 10
5.5 Closure and meta-rank growth . . . 11

6 The exact Contender 20 definition **11**
6.1 The predecessor family . . . 11
6.2 A self-sized schedule and two self-sized ranks . . . 11
6.3 Initial lift, third-order lift, and final family . . . 12
6.4 Final reflection and closed number . . . 12

7 Formal domination chain **13**
7.1 The initial diagonal lift is good . . . 13
7.2 Architectural domination . . . 13
7.3 The predecessor family is beaten inside the hyper-family . . . 13
7.4 The old raw reflection is also below the new family . . . 14
7.5 Programs relative to the final family . . . 14
7.6 The closed comparison . . . 14

8 Constructivity and executability **14**
8.1 Only structural recursion is introduced . . . 15
8.2 No new axiom is needed by design . . . 15
8.3 The no-laziness condition . . . 15
8.4 Why the number should never be evaluated . . . 15

9 Why the increase is substantial **15**
9.1 A new reusable type-level closure . . . 15
9.2 The accepted hierarchy becomes the base object . . . 16
9.3 The schedule is used at two nested levels . . . 16
9.4 The final object language sees the whole generated family . . . 16

10 Heuristic growth discussion, with explicit limits **16**
10.1 What can be said safely . . . 16
10.2 What is not claimed . . . 16

11 Lean engineering and replay plan **17**
11.1 Modules . . . 17
11.2 Automated acceptance sequence . . . 17
11.3 Likely elaboration pressure points . . . 18

12 Static audit and evidence discipline **18**

13 Further research directions **18**
13.1 Generic higher-type reflection towers . . . 18
13.2 Well-founded rather than finite meta-ranks . . . 19
13.3 Collapsing-function atom orders . . . 19
13.4 A semantic calibration theorem . . . 19
13.5 Proof automation for explicit-occurrence arguments . . . 19
13.6 Separating schedule, inner rank, and meta-rank . . . 19
13.7 Internal final-family reflection at the lift level . . . 19

14 Conclusion **20**

A Complete executable definitions **20**

B Complete proof modules **22**

C Focused kernel audit **28**

D Acceptance checklist **29**

1 Problem, baseline, and research standard

The large-number game becomes mathematically substantive when two shortcuts are forbidden. First, the proposed number must be computable rather than defined through an uncomputable extremal function such as Busy Beaver. Second, a larger-number claim must be a theorem rather than an informal appeal to a notation's intended strength. The original contest of Roux used Coq; BusyLean ports the game to Lean 4 and has developed a long sequence of successively stronger reflective constructions [2, 3, 1].

The package was prepared against the audited BusyLean `main` commit `64d31a0cf6f1aa4477a088be5554957de6e87a61`. The pinned toolchain is `leanprover/lean4:v4.34.0`. At that head, Contender 19 is the accepted entry. Its defining equations are, schematically,

$$\begin{aligned} \mathrm{family}_{18}(n) &:= F_{18}(F_{18}(n)), \\ F_{19}(n) &:= \mathrm{diagonalLift}(S_{19}(n), r_{19}(n), L, \mathrm{family}_{18}, n, n), \\ \mathsf{C}_{19} &:= F_{19}(F_{19}(42)), \end{aligned}$$

where L is large-Veblen-ladder reflection, $S_{19}(n)$ is a self-sized well-founded schedule, and $r_{19}(n) = \mathrm{family}_{18}(n) + 1$.

1.1 Contest constraints

A proposal intended for integration must satisfy more than mathematical plausibility. The current repository rules require the following.

1. The entry is a closed, computable Lean `Nat`. Definitions using `noncomputable`, `partial`, `unsafe`, opaque executable stand-ins, foreign implementations, or uncomputable oracles are excluded.
2. A theorem proves strict domination of the accepted predecessor.
3. Kernel dependencies are restricted to `propext` and `Quot.sound`; in particular, `Classical.choice`, custom axioms, `sorry`, and `native_decide` are excluded.
4. With earlier contenders precompiled, the new definition is allotted 15 seconds and the new proof phase 60 seconds on a reasonable machine.
5. The definition may use the predecessor's *construction* as a seed, but it may not simply mention the predecessor's closed constant and add or iterate something trivial.
6. The kernel, elaborator, and build toolchain may not be exploited.

These constraints shape the mathematics. A valid improvement needs a total higher-order construction whose strictness follows from explicit finite maxima or visible recursive occurrences, rather than from an unproved semantic monotonicity principle.

1.2 Meaning of "significantly exceeds"

Every legal successor must be strictly larger, but a research-quality proposal should expose a qualitative increase. This package adopts the following standard.

Definition 1.1 (Architectural significance)**.** A successor construction is architecturally significant when it introduces a new reusable closure principle and proves a strict domination theorem for an entire predecessor-style operator, not merely for one closed numerical output.

Contender 20 meets this standard through the theorem `diagonalLift_lt_lift20`: at fixed input, its completed third-order lift dominates a complete current-style diagonal hyperlift instantiated with the same stronger schedule and inner rank. The final number comparison is a corollary of that architecture plus explicit reflection maxima.

Validation warning 1.2. The Lean code in this archive is a carefully matched proof draft, not a claimed kernel-checked result. The packaging environment had no `lean` or `lake` executable. The included `check20.py` performs the mandatory replay, axiom closure, dependency closure, and timing tests in a proper BusyLean checkout.

2 The accepted machinery through Contender 19

Understanding the proposed increase requires separating the repository's *object-language axis* from its *amplifier axis*.

2.1 Families and transformers

A countable oracle family has two natural-number arguments:

$$\mathsf{Family} := \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

The first argument selects an oracle coordinate and the second is the numerical input. A transformer maps such families to such families:

$$\mathsf{Transformer} := \mathsf{Family} \to \mathsf{Family}.$$

BusyLean's primitive reflective transformer evaluates a bounded collection of programs relative to a packed oracle family and returns a number strictly above all visible oracle values and all bounded program values. A transformer T is called progressive when

$$i \le n \quad \Longrightarrow \quad s(i,n) < T(s)(o,n)$$

for every seed family s and output coordinate o.

The definition is deliberately coordinate-sensitive. Progressivity is not a monotonicity assertion of the form $s \le t \Rightarrow T(s) \le T(t)$. Reflective programs can inspect and branch on oracle values, so semantic monotonicity is in general false. BusyLean's proofs instead track a selected child or a selected coordinate that literally occurs in the next maximum.

2.2 Autonomous lifting

A *transformer lift* acts on transformers:

$$\mathsf{TransformerLift} := \mathsf{Transformer} \to \mathsf{Transformer}.$$

The repository's `liftTransformer` iterates a progressive transformer along a universal well-founded W-tree. Its strictness theorem has the form

$$T(s)(i,n) < \mathrm{liftTransformer}(T)(s)(o,n), \qquad i \le n.$$

The proof follows a concrete path through the universal tree. It does not compare arbitrary oracles.

The function `stageLift` accepts an arbitrary transformer lift A, a base transformer T, and a W-tree a. At a leaf it applies A to T; at a node it recursively builds one child transformer per branch, splices those children coordinatewise, and applies A to the splice. Thus `stageLift` raises tree reflection from families to transformers.

2.3 Diagonal hyperreflection

For a fixed schedule S, the accepted `diagonalLift` is a natural hierarchy of transformer lifts:

$$D_0^S(T) := \text{liftTransformer}(T),$$
$$D_{k+1}^S(T) := \text{stageLift}(D_k^S, D_k^S(T), S).$$

The successor uses the previous rank twice: as the local lift at every schedule node and as a pre-amplification of the base. BusyLean proves two crucial properties for every k:

1. D_k^S preserves progressive transformers;
2. D_k^S is strictly inflationary on every progressive transformer.

The rank is not a cosmetic counter. The theorem `diagonalLift_rank_lt_succ` proves strict increase from rank k to rank $k+1$ at every visible coordinate.

2.4 The large-Veblen object language

The object-language component of Contender 19 is independent of the amplifier. Its atom order consists of normal Schütte-style Klammersymbole whose Veblen argument positions are themselves terms. The repository constructs a universe for every well-founded atom order and proves strong normalization of the universe-ladder language. A bounded evaluator `lvalue` therefore returns a natural number for every bounded, type-checked large-Veblen-ladder program. The family reflector `reflectLFamily` is explicitly larger than

- every visible family coordinate $s(i, n)$ with $i \leq n$; and
- every value of a bounded large-Veblen-ladder program run relative to the packed family s.

This is the base transformer `lTransformer`. Contender 19 applies a self-ranked diagonal lift to it. Contender 20 keeps this powerful object language and raises the amplifier by one more type level.

3 Design alternatives and the chosen increase

Several superficially easy successors were rejected as insufficiently informative.

Only increase a numeral. Replacing a fixed constant by a larger fixed constant, or composing the final function one additional time, would prove a larger number but add no closure principle. It would also violate the spirit of the no-laziness rule.

Only increase the existing rank. Setting the diagonal rank to an even larger expression would remain inside the same natural hierarchy D_k^S. That is a quantitative increase, but it does not dominate the hierarchy as an operator in a new way.

Only lengthen the ordinal ladder. A collapsing-function notation beyond the large Veblen ordinal would be a valuable future object-language advance, but a correct normal-form system and well-foundedness proof are substantial projects. Raw collapsing terms often admit descending chains unless normality and side conditions are designed with great care. A rushed notation would weaken rather than strengthen the formal package.

Only reseed Contender 19. Using $n \mapsto F_{19}(F_{19}(n))$ as a seed is legal and useful, but reseeding alone does not explain a new amplifier.

The chosen construction uses the accepted diagonal lift as its *initial object*, then treats entire transformer lifts as branchable, spliceable data. This is exactly the next simple type over the accepted amplifier:

$$\mathsf{LiftTransformer} := \mathsf{TransformerLift} \to \mathsf{TransformerLift}.$$

The new closure acts at this level, and only afterward returns to a numerical family and performs one final object-language reflection.

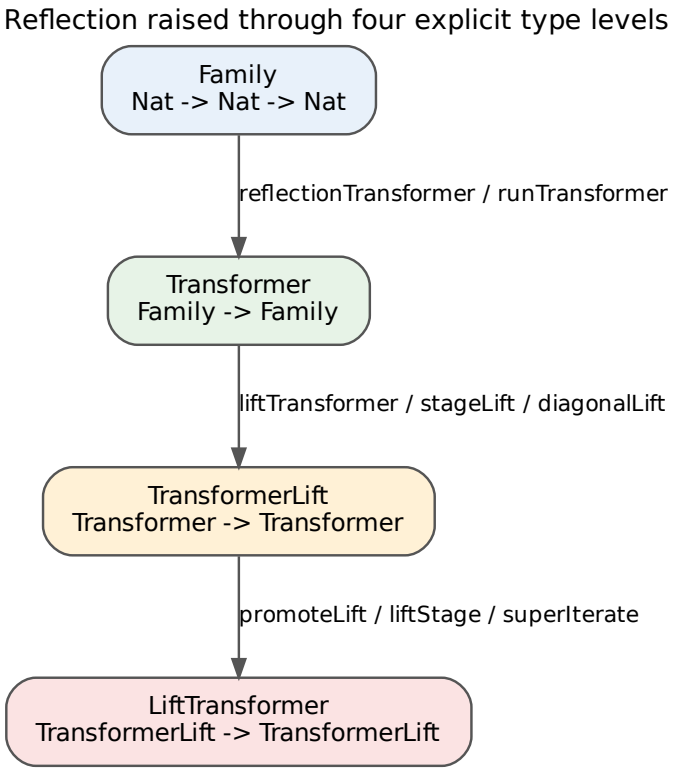


Figure 1: The new construction raises reflection one explicit simple-type level. The labels describe Lean abbreviations, not an informal analogy.

4 Third-order autonomous hyperreflection

This section gives the mathematical content of `SuperReflection.lean`. The adjective "third-order" refers to the explicit hierarchy

$$\mathsf{Family}, \qquad \mathsf{Family} \to \mathsf{Family}, \qquad (\mathsf{Family} \to \mathsf{Family}) \to (\mathsf{Family} \to \mathsf{Family}),$$
$$\mathsf{TransformerLift} \to \mathsf{TransformerLift}.$$

It is not a claim about the order of arithmetic or a proof-theoretic ordinal.

4.1 Splicing whole lifts

Given a countable family of transformer lifts $A_0, A_1, \ldots$, define a new lift by

$$\mathrm{spliceLifts}(A)(T) := \mathrm{spliceTransformers}(i \mapsto A_i(T)).$$

At output coordinate o, the splice selects $A_o(T)$ and evaluates it at its canonical output coordinate 0. This definition is important for two reasons. First, it makes a family of complete amplifiers dynamically selectable. Second, it preserves the explicit-coordinate proof discipline: coordinate o witnesses the occurrence of the oth child lift.

Listing 1: Coordinatewise splicing at the new type level.

```
abbrev LiftTransformer := TransformerLift → TransformerLift

def spliceLifts (lifts : Nat → TransformerLift) : TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)
```

4.2 Promotion internalizes one diagonal successor

For a fixed schedule S, define

$$\mathrm{promoteLift}_S(A)(T) := \mathrm{stageLift}(A, A(T), S).$$

This is precisely the shape of one successor in the accepted diagonal hierarchy, but abstracted as an operation on arbitrary lifts. If $A = D_k^S$, then promotion recovers D_{k+1}^S extensionally at the defining equation. The abstraction is what allows the successor operation itself to be iterated along a W-tree.

Listing 2: One accepted hyperreflection successor, abstracted as a lift transformer.

```
def promoteLift (schedule : WTree Nat) : LiftTransformer :=
  fun lift base => stageLift lift (lift base) schedule
```

4.3 Tree recursion over lift transformers

Let M : LiftTransformer and A : TransformerLift. Define $\mathrm{liftStage}(M, A, a)$ by recursion on a W-tree a:

$$\begin{aligned} \mathrm{liftStage}(M, A, \mathsf{leaf}) &:= M(A), \\ \mathrm{liftStage}(M, A, \mathsf{node}(f)) &:= M\left(\mathrm{spliceLifts}(i \mapsto \mathrm{liftStage}(M, A, f(i)))\right). \end{aligned}$$

This is the exact next-type analogue of `stageLift`. The tree does not merely choose numbers or transformers; its branches contain complete transformer lifts.

Listing 3: Well-founded tree reflection over complete transformer lifts.

```
def liftStage (amp : LiftTransformer) (base : TransformerLift) :
    WTree Nat → TransformerLift
  | .leaf => amp base
  | .node f =>
      amp (spliceLifts (fun i => liftStage amp base (f i)))
```

4.4 Autonomous closure and finite meta-rank

The autonomous closure uses the same schedule in two roles:

$$\mathrm{superClosure}_S(A) := \mathrm{liftStage}(\mathrm{promoteLift}_S, A, S).$$

Internally, each application of `promoteLift` traverses S through `stageLift`; externally, `liftStage` branches over whole lifts along S again. This is a genuine nested closure, not a second name for one rank increment.

Finally define finite iteration:

$$\begin{aligned}\mathrm{superIterate}_S(0, A) &:= A,\\ \mathrm{superIterate}_S(k+1, A) &:= \mathrm{superClosure}_S(\mathrm{superIterate}_S(k, A)).\end{aligned}$$

The recursion parameter is a natural number. Therefore the operation remains total and executable in principle even when the chosen rank is unimaginably large.

Listing 4: Autonomous third-order closure and its finite meta-rank hierarchy.

```
def superClosure (schedule : WTree Nat) (base : TransformerLift) :
    TransformerLift :=
  liftStage (promoteLift schedule) base schedule

def superIterate (schedule : WTree Nat) :
    Nat → TransformerLift → TransformerLift
  | 0, base => base
  | k + 1, base => superClosure schedule (superIterate schedule k base)
```

5 The invariant that makes comparison possible

Higher-order reflection is easy to define and easy to overclaim. The central technical contribution is a small invariant strong enough to propagate through every new constructor and weak enough to be proved without false monotonicity.

5.1 Two accepted properties

For a transformer lift A : TransformerLift, BusyLean already defines:

Definition 5.1 (Preservation)**.** A preserves progressivity if every progressive transformer T is mapped to a progressive transformer $A(T)$.

Definition 5.2 (Strict inflationarity)**.** A is strictly inflationary if, whenever T is progressive and $i \le n$,

$$T(s)(i, n) < A(T)(s)(o, n)$$

for every family s and output coordinate o.

We package them.

Definition 5.3 (Good lift)**.**

$$\mathsf{GoodLift}(A) :\Longleftrightarrow \mathrm{PreservesProgressive}(A) \wedge \mathrm{StrictlyInflationary}(A).$$

A lift transformer M : LiftTransformer preserves good lifts when $\mathsf{GoodLift}(A) \Rightarrow \mathsf{GoodLift}(M(A))$. It is lift-strict when, for every good A and every progressive T,

$$A(T)(s)(i, n) < M(A)(T)(s)(o, n) \qquad (i \le n).$$

5.2 Splicing preserves the invariant

Lemma 5.4 (`spliceLifts_good`)**.** *If every A_j is a good lift, then* $\mathrm{spliceLifts}(j \mapsto A_j)$ *is a good lift.*

Proof idea. For preservation, fix a progressive base transformer T. Every $A_j(T)$ is progressive, and the accepted theorem `spliceTransformers_progressive` then applies. For strictness, output coordinate o of the splice is definitionally coordinate 0 of $A_o(T)$. Apply strictness of A_o to the visible seed coordinate i. No comparison between different seed families is needed. □

5.3 Promotion is good and lift-strict

Lemma 5.5 (`promoteLift_good`)**.** *For every schedule S, if A is good, then* $\mathrm{promoteLift}_S(A)$ *is good.*

Proof idea. Preservation is the accepted theorem `stageLift_progressive`, applied to the progressive transformer $A(T)$. For strictness, chain two explicit inequalities:

$$T(s)(i,n) < A(T)(s)(i,n) < \mathrm{stageLift}(A, A(T), S)(s)(o,n).$$

The first is strict inflationarity of A; the second is the accepted `base_lt_stageLift` theorem, with base transformer $A(T)$. □

Lemma 5.6 (`lift_lt_promoteLift`)**.** *If A is good and T is progressive, then*

$$A(T)(s)(i,n) < \mathrm{promoteLift}_S(A)(T)(s)(o,n)$$

for every $i \le n$.

This second statement is stronger than what is required to show that promotion is good: it says promotion strictly exceeds the input lift itself. It becomes the local strictness hypothesis for the next tree recursion.

5.4 The node argument one level higher

Theorem 5.7 (`baseLift_lt_liftStage`)**.** *Suppose M preserves good lifts and is lift-strict. If A is good and T is progressive, then for every W-tree a and every visible coordinate $i \le n$,*

$$A(T)(s)(i,n) < \mathrm{liftStage}(M, A, a)(T)(s)(o,n).$$

Proof architecture. At a leaf, this is exactly lift-strictness of M.

At a node $\mathsf{node}(f)$, first follow child zero recursively:

$$A(T)(s)(i,n) < \mathrm{liftStage}(M, A, f(0))(T)(s)(0,n).$$

By the preceding preservation theorem, every child lift is good, so their coordinatewise splice is good. The value of child zero at output coordinate zero is definitionally coordinate zero of that splice. Lift-strictness of M therefore gives

$$\mathrm{liftStage}(M, A, f(0))(T)(s)(0,n) < M(\mathrm{spliceLifts}(\ldots))(T)(s)(o,n),$$

which is the parent value. Transitivity completes the proof.

The argument is notable for what it does *not* use: no transitivity of an ordinal notation order, no monotonicity of program evaluation in an oracle, and no global comparison of branch families. Only an explicitly selected child and an explicitly selected coordinate are used. □

5.5 Closure and meta-rank growth

Specializing M to promoteLift$_S$ yields the following.

Theorem 5.8 (Autonomous closure)**.** *If A is good, then* superClosure$_S(A)$ *is good and strictly exceeds A on every progressive base transformer and every visible coordinate.*

Natural-number induction then gives:

Theorem 5.9 (`superIterate_good`)**.** *For every k, if A is good, then* superIterate$_S(k, A)$ *is good.*

Theorem 5.10 (`superIterate_step_lt`)**.** *For every k, progressive T, and visible coordinate $i \le n$,*

$$\text{superIterate}_S(k, A)(T)(s)(i, n) < \text{superIterate}_S(k+1, A)(T)(s)(o, n).$$

Corollary 5.11 (`lift_lt_superIterate_succ`)**.** *Every positive meta-rank strictly exceeds the initial lift A.*

This corollary is the formal reason the new meta-rank is extensionally meaningful. It is also the bridge to the architectural domination theorem for Contender 20.

6 The exact Contender 20 definition

The new construction uses the accepted family, but not the accepted closed constant.

6.1 The predecessor family

Define

$$\text{family}_{19}(n) := F_{19}(F_{19}(n)).$$

At $n = 42$ this is definitionally Contender 19, but as a function it is available at every input. The seed family repeats it at every coordinate:

$$\text{seed}_{19}(i, n) := \text{family}_{19}(n).$$

Using the preceding construction as a family rather than importing the closed constant follows the established BusyLean pattern and satisfies the no-laziness rule.

6.2 A self-sized schedule and two self-sized ranks

Let

$$S_{20}(n) := \text{realize}(\text{hyper}(\text{family}_{19}(n) + 42, \text{encode}(\text{tree}_{12}))).$$

This is a computable W-tree. Define both ranks by

$$r_{\text{in}}(n) = r_{\text{meta}}(n) := \text{family}_{19}(n) + 1.$$

Keeping the two names separate is deliberate. The inner rank belongs to the accepted diagonal hierarchy; the meta-rank counts autonomous closures of whole lifts. Future variants may separate their growth rates.

6.3 Initial lift, third-order lift, and final family

The initial lift is

$$A_n := \mathrm{diagonalLift}(S_{20}(n), r_{\mathrm{in}}(n)).$$

The third-order lift is

$$L_{20}(n) := \mathrm{superIterate}(S_{20}(n), r_{\mathrm{meta}}(n), A_n).$$

Applying this lift to the large-Veblen transformer L and the seed family gives an entire output family:

$$H_{20}(n) := L_{20}(n)(L)(\mathrm{seed}_{19}).$$

In Lean, $H_{20}(n)$ has type `Family`; it is not prematurely collapsed to one selected output coordinate.

6.4 Final reflection and closed number

The numerical function is a final large-Veblen reflection over the complete third-order family:

$$F_{20}(n) := \mathrm{reflectLFamily}(H_{20}(n), n).$$

Finally,

$$\boxed{\mathsf{C}_{20} := F_{20}(F_{20}(42)).}$$

Listing 5: The complete numerical definition.

```
def family19 (n : Nat) : Nat := F19 (F19 n)

def seed19 : Family := fun _ => family19

def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42) (Autonomous.encode tree12))

def innerRank20 (n : Nat) : Nat := family19 n + 1

def metaRank20 (n : Nat) : Nat := family19 n + 1

def lift20 (n : Nat) : TransformerLift :=
  superIterate (schedule20 n) (metaRank20 n)
    (diagonalLift (schedule20 n) (innerRank20 n))

def hyperFamily20 (n : Nat) : Family :=
  lift20 n lTransformer seed19

def F20 (n : Nat) : Nat :=
  reflectLFamily (hyperFamily20 n) n

def contender_20 : Nat := F20 (F20 42)
```

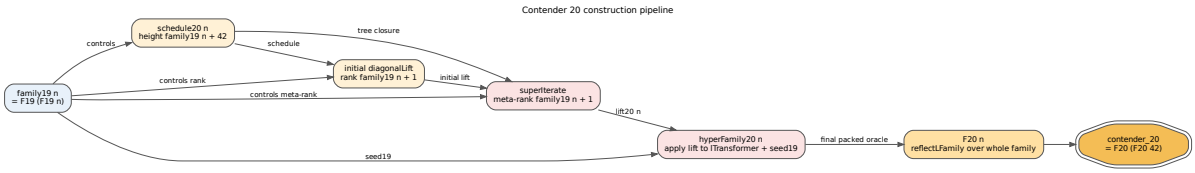


Figure 2: The construction deliberately preserves the complete third-order family until the final object-language reflection.

7 Formal domination chain

The proposed proof has two levels: a reusable operator theorem and the closed number theorem required by the contest.

7.1 The initial diagonal lift is good

The accepted theorems immediately give

$$\mathsf{GoodLift}(A_n).$$

In Lean this is `diagonalLift20_good`, a pair consisting of `diagonalLift_preservesProgressive` and `diagonalLift_strictlyInflationary`. Applying `superIterate_good` then gives $\mathsf{GoodLift}(L_{20}(n))$.

7.2 Architectural domination

Because $r_{\mathrm{meta}}(n) = \mathrm{family}_{19}(n) + 1$ is positive, `lift_lt_superIterate_succ` yields the main theorem.

Theorem 7.1 (`diagonalLift_lt_lift20`)**.** *For every input n, progressive transformer T, seed family s, and $i \le n$,*

$$A_n(T)(s)(i,n) < L_{20}(n)(T)(s)(o,n).$$

This theorem compares complete lifts under arbitrary progressive bases and arbitrary seed families. It does not merely compare the two closed contenders. Moreover, A_n is a current-style diagonal lift using the *stronger* Contender 20 schedule and the *stronger* Contender 20 inner rank. The new meta-layer strictly exceeds that whole object before the final reflection.

7.3 The predecessor family is beaten inside the hyper-family

Since $L_{20}(n)$ preserves progressivity and `lTransformer` is progressive, the output transformer $L_{20}(n)(L)$ is progressive, where L is `lTransformer`. Apply it to `seed19` and select input coordinate zero:

$$\mathrm{family}_{19}(n) = \mathrm{seed}_{19}(0,n) < H_{20}(n)(0,n).$$

This is `family19_lt_hyperFamily20`.

The final reflector contains an explicit prefix maximum over the visible coordinates of $H_{20}(n)$. Hence

$$H_{20}(n)(0,n) < F_{20}(n).$$

Combining the two inequalities gives the pointwise theorem

$$\mathrm{family}_{19}(n) < F_{20}(n). \tag{1}$$

7.4 The old raw reflection is also below the new family

Strict inflationarity of $L_{20}(n)$, with base transformer `lTransformer`, gives

$$\mathrm{reflectLFamily}(\mathrm{seed}_{19}, n) < H_{20}(n)(0, n).$$

The final reflection then gives

$$\mathrm{reflectLFamily}(\mathrm{seed}_{19}, n) < F_{20}(n).$$

This is useful evidence that the increase is not obtained solely by changing the seed: even one complete large-Veblen reflection over the old seed lies strictly below the new third-order family before the final reflection.

7.5 Programs relative to the final family

The final layer was chosen so that the strongest program bound concerns the final oracle family, not merely the old seed.

Theorem 7.2 (`final_lvalue_lt_F20`)**.** *If t is an enumerated large-Veblen-ladder program of depth at most n, then*

$$\mathrm{lvalue}(\mathrm{packFamily}(H_{20}(n)), n, t) < F_{20}(n).$$

The proof is direct from the maximum built into `reflectLFamily`. It states that the object language may inspect the complete third-order family through the computable family packing, yet every bounded program value remains below the final output.

7.6 The closed comparison

The accepted input-growth theorem gives

$$n < F_{19}(n) < F_{19}(F_{19}(n)) = \mathrm{family}_{19}(n).$$

Together with Equation (1), this gives $n < F_{20}(n)$. Now set $n = 42$ in the family domination theorem:

$$\mathsf{C}_{19} = \mathrm{family}_{19}(42) < F_{20}(42).$$

Apply input growth at $F_{20}(42)$:

$$F_{20}(42) < F_{20}(F_{20}(42)) = \mathsf{C}_{20}.$$

By transitivity:

Theorem 7.3 (Required comparison)**.**

$$\boxed{\mathsf{C}_{19} < \mathsf{C}_{20}.}$$

The corresponding Lean declaration is `contender_19_lt_contender_20`*.*

8 Constructivity and executability

The values in this contest are intentionally infeasible to evaluate. That is not a defect: computability means that a finite procedure exists, not that the procedure terminates within physical resources. The proposed definition is computable for the following concrete reasons.

8.1 Only structural recursion is introduced

The new executable definitions recurse on exactly two inductive objects.

- `liftStage` recurses structurally on a value of type `WTreeNat`. Every recursive call is made on an immediate child.
- `superIterate` recurses structurally on a natural-number rank. The successor case calls the function at the predecessor rank.

The schedule itself is produced by the existing computable `Autonomous.realize`. The large-Veblen evaluator uses the existing strong-normalization proof to define a total normalizer. The final reflector uses maxima over finite bounded enumerations. No semantic search for a proof or halting witness appears in the executable path.

8.2 No new axiom is needed by design

The new definitions are axiom-free executable terms. The proofs are assembled from accepted progressivity and strictness theorems, induction, definitional reduction, and transitivity of natural-number order. Therefore no use of choice is mathematically expected. Nonetheless, Lean's actual dependency closure is the authority; `Audit20.lean` prints the axiom sets of the new key declarations, and `check20.py` rejects every axiom outside `propext` and `Quot.sound`.

8.3 The no-laziness condition

The source definition of `contender_20` does not mention the constant `contender_19`. It imports and uses F_{19}, the previous construction, to define a seed function at arbitrary inputs. This is precisely the distinction allowed by the contest rules. The focused audit additionally checks the transitive constant closure of `contender_20` against all earlier contender constants.

8.4 Why the number should never be evaluated

Even very small fragments of the object-language enumerations grow rapidly, and the proposed rank is itself controlled by a twice-iterated predecessor family. Actual evaluation would attempt to realize nested W-tree schedules, repeated higher-order closures, and bounded normalizations at impossible sizes. Lean needs only to type-check the definitions and proofs; it does not normalize the closed natural number. The package therefore contains no `#eval` of any contender.

9 Why the increase is substantial

The strict inequality theorem alone does not communicate the construction's scale. Four structural features distinguish Contender 20 from a cosmetic successor.

9.1 A new reusable type-level closure

Contender 19 chooses a large natural rank in a hierarchy of transformer lifts. Contender 20 makes transformer lifts themselves the branch objects of a well-founded reflection. A node can splice a countable family of complete lifts, and the next operation can inspect that splice as one lift. This capability cannot be represented by merely increasing the natural rank of one fixed `diagonalLift` definition without inlining a new higher-order recursion.

9.2 The accepted hierarchy becomes the base object

The initial object A_n is not rank zero or a weak primitive. It is already a self-ranked current-style diagonal lift with rank $\text{family}_{19}(n) + 1$ and schedule height controlled by the same predecessor family. The positive meta-rank then strictly dominates this entire initial object.

9.3 The schedule is used at two nested levels

Within each promotion, `stageLift` traverses $S_{20}(n)$. Around those promotions, `liftStage` traverses the same schedule while branching over whole lifts. Thus each autonomous closure nests schedule-driven reflection rather than repeating a fixed unary operation.

9.4 The final object language sees the whole generated family

The third-order lift is not immediately sampled at one coordinate. It produces $H_{20}(n) : \mathsf{Family}$, which is packed as the oracle of one final large-Veblen-ladder reflection. The final program bound therefore includes programs that dynamically query coordinates of the generated third-order family. This interaction between amplifier and object language is stronger than appending an arithmetic successor to the amplified value.

10 Heuristic growth discussion, with explicit limits

Fast-growing-hierarchy notation is useful for orientation, but it can be misleading when a construction mixes impredicative type systems, autonomous tree reflection, universe ladders, and higher-type operator closure. The formal result of this package is a collection of natural-number inequalities, not an exact ordinal calibration.

10.1 What can be said safely

At a qualitative level, the accepted hierarchy $k \mapsto D_k^S$ is converted from a parameterized family of lifts into the input of a new autonomous closure operator. One application of `superClosure` contains:

1. an outer traversal of S whose branches are complete transformer lifts;
2. coordinatewise splicing of those lift branches;
3. at each outer node, a promotion that itself performs an inner `stageLift` traversal of S; and
4. pre-amplification of every base transformer by the local lift.

Then this closure is iterated through a rank controlled by the full predecessor family, and the resulting family is reflected once more by the large-Veblen object language. This is compelling structural evidence of a large gap beyond Contender 19, in addition to the formal strict inequality.

10.2 What is not claimed

This article does not identify a proof-theoretic ordinal α such that F_{20} is equivalent to a standard fast-growing function f_α. It does not claim that one higher simple type automatically corresponds to a particular ordinal operation. It does not compare the construction with arbitrary future collapsing-function ladders. Such statements would require a semantic ordinal analysis of the exact reflective languages and higher operators, not merely an inspection of source syntax.

The distinction follows the methodological lesson of transfinite reflection progressions: iteration principles can be calibrated only after the underlying theory, notation system, fundamental sequences, and reflection schema are specified precisely [4, 5, 6]. Here the kernel-checked inequalities are precise; the scale discussion remains a research heuristic.

11 Lean engineering and replay plan

The package is organized as an overlay so it can be reviewed without modifying the repository prematurely.

11.1 Modules

File	Purpose
`BusyLean/SuperReflection.lean`	New executable type-level operations: `LiftTransformer`, `spliceLifts`, `promoteLift`, `liftStage`, `superClosure`, and `superIterate`.
`BusyLean/SuperReflectionFacts.lean`	The `GoodLift` invariant, preservation and strictness lemmas, tree-node comparison, and meta-rank growth.
`BusyLean/Contender20.lean`	The self-sized schedule, inner and meta ranks, final family, F_{20}, and the closed number.
`BusyLean/Comparison20.lean`	Architectural domination, pointwise family domination, program bound, input growth, and the required closed comparison.
`test/Audit20.lean`	Focused `#print axioms`, transitive constant-closure rejection, and declaration checks.
`scripts/check20.py`	Clean build, 15/60-second budget checks, audit execution, and axiom parser.

11.2 Automated acceptance sequence

After applying the overlay, the intended command is

```
python3 scripts/check20.py
```

It performs the following phases from deleted build directories.

1. Build `BusyLean.Comparison19` as the accepted, uncharged baseline.
2. Build `BusyLean.Contender20` and reject a time above 15 seconds.
3. Build `BusyLean.Comparison20` and reject a time above 60 seconds.
4. Run `test/Audit20.lean`.
5. Parse all `#print axioms` reports and reject any axiom outside the permitted pair.
6. Reject the build if the constant closure of `contender_20` contains any earlier contender constant.

11.3 Likely elaboration pressure points

Although the proof draft mirrors accepted signatures, honest engineering requires identifying where replay may expose a mismatch.

1. The `change` step in the node case relies on definitional reduction of `spliceLifts` and `spliceTransformers` at coordinate zero. If elaboration does not unfold far enough, a local `simpa[spliceLifts,spliceTransformers]` should repair the step without changing the theorem.
2. The final comparison relies on definitional equality between `family1942` and `contender_19`. A replay may request an explicit `simpa[family19,contender_19,contender_20]`.
3. Natural-number expressions $(k + 1) + 1$ and $(\operatorname{succ} k) + 1$ are definitionally close but may require a one-line simplification in the induction on positive meta-ranks.
4. The definition budget should be comfortable because the new executable modules are small and predecessor modules are precompiled, but only the target machine can establish the timing result.

These are ordinary elaboration risks, not gaps in the mathematical strategy. The package records them so that any repair remains minimal and auditable.

12 Static audit and evidence discipline

A source audit cannot replace the kernel, but it can prevent avoidable submission errors. The included `static_audit.py` strips nested Lean comments and strings, then checks the four library modules for forbidden implementation words and elaboration-time commands. It checks expected declarations, verifies that every line is at most 100 columns, confirms that the definition source contains F_{19} but not the closed constant `contender_19`, and records SHA-256 hashes.

The audit report is intentionally labeled `static_source_audit_not_kernel_replay`. It reports `kernel_replay_performed: false`. The claim matrix classifies each statement as one of:

- verified by source or PDF tooling;
- proof drafted but kernel replay pending;
- entirely unverified until timing or axiom audit runs.

This distinction is part of the research result. Large-number proposals are particularly vulnerable to impressive notation masking an unchecked premise. The archive therefore does not convert a missing toolchain into an implicit claim of success.

13 Further research directions

The proposal opens several concrete lines of work.

13.1 Generic higher-type reflection towers

The progression

$$\mathsf{Family}, \quad \mathsf{Family} \to \mathsf{Family}, \quad \mathsf{TransformerLift}, \quad \mathsf{LiftTransformer}$$

can be abstracted using an indexed family of simple types and a generic notion of coordinatewise splicing. One could define a finite tower whose height is itself input-controlled. The main challenge is not the executable recursion but a uniform invariant that specializes to progressivity and strict inflationarity at each level without generating unmanageable dependent casts.

13.2 Well-founded rather than finite meta-ranks

The accepted repository already builds universe ladders over arbitrary well-founded atom orders. A natural next question is whether `superIterate` can be generalized from $\mathbb{N}$ to an explicit well-founded notation type. At a limit notation, the operation would need to splice the lower-rank lifts in a way that preserves `GoodLift`. The current `spliceLifts_good` theorem is exactly the local lemma such a construction would require. A meaningful proposal would need an explicit fundamental-sequence or predecessor enumeration and a strict comparison theorem for every accessible notation.

13.3 Collapsing-function atom orders

The object-language axis can be advanced independently. Extending the ladder beyond the large Veblen ordinal with Buchholz-style or Takeuti-style collapsing functions is attractive, but raw terms are not automatically well-founded under the intended comparison. The research task is to design normal forms, side conditions, and a Boolean comparison whose case split supports a Lean accessibility proof. The successful sorted-list, Veblen, and Klammersymbol proof patterns suggest that a carefully restricted notation system may be tractable.

13.4 A semantic calibration theorem

The present article proves only internal inequalities. A deeper project would interpret the reflective families into a standard ordinal-indexed fast-growing hierarchy and prove lower or upper bounds. This requires a semantics for the W-tree schedules, transformer lifts, and lift transformers. Even a theorem for a simplified fragment—for example, finite trees and a primitive progressive transformer—would clarify which ordinal operations correspond to the higher type closures.

13.5 Proof automation for explicit-occurrence arguments

The accepted reflection proofs repeatedly follow a selected child, expose a coordinate of a splice, and apply a strictness invariant. A small Lean tactic or theorem library could automate this pattern. Such automation would reduce the risk of coordinate mistakes and make fourth- and fifth-type-level experiments easier to audit.

13.6 Separating schedule, inner rank, and meta-rank

Contender 20 sets both ranks to $\mathrm{family}_{19}(n) + 1$ and derives the schedule from the same family. This symmetry is clean but not necessarily optimal. One can investigate asymmetric choices, such as using the generated inner family to control the meta-rank, provided the resulting definition remains nonrecursive and its comparison proof exposes a finite witness rather than relying on monotonicity.

13.7 Internal final-family reflection at the lift level

The present final step returns to the large-Veblen object language. A stronger higher-type analogue would package a countable family of third-order lifts and perform one additional lift-level reflection before numerical evaluation. The current `spliceLifts` and `GoodLift` infrastructure provides part of the interface, but a new autonomous universal index for lift-producing programs would be needed to justify the adjective “self-referential” at that level.

14 Conclusion

Contender 20 is designed around a simple principle: once a reflective hierarchy has been formalized as an operator, a meaningful successor should reflect over the operator itself rather than merely choose a larger parameter. BusyLean's accepted Contender 19 already provides a strong object language and a natural hierarchy of transformer lifts. The proposed construction raises that hierarchy one explicit type level, branches over complete lifts along an autonomous W-tree, iterates the resulting closure through a self-sized meta-rank, preserves the entire output family, and performs one final large-Veblen reflection over it.

The invariant `GoodLift` makes the construction comparably disciplined to the accepted lower levels. Every strict inequality is witnessed by an explicit coordinate or child. The operator theorem `diagonalLift_lt_lift20` formalizes the qualitative gain, while the pointwise family theorem and input growth yield the required `contender_19_lt_contender_20`. The number is closed and computable in principle, and the package includes the artifacts needed for repository integration and independent scrutiny.

The remaining step is mechanical but essential: replay the sources with Lean 4.34.0, run the axiom and dependency audit, and measure the definition and proof phases. Until that replay succeeds, this is a serious, testable research proposal rather than an accepted formal theorem.

A Complete executable definitions

Listing 6: `BusyLean/SuperReflection.lean`

```
import BusyLean.HyperReflection

/-!
# Third-order autonomous hyperreflection

The accepted hyperreflection layer acts on `Transformer`s through
`TransformerLift = Transformer → Transformer`.  This module raises the
construction one more type level.

A `LiftTransformer` maps transformer lifts to transformer lifts.  We can splice
a countable family of lifts coordinatewise, promote a lift by one ordinary
hyperreflection step, iterate an arbitrary lift transformer along a W-tree, and
then iterate that whole autonomous closure through a finite meta-rank.

Every definition is structural recursion on `Nat` or `WTree Nat`.  There is no
partiality, oracle, or evaluation-time metaprogramming.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- One type level above `TransformerLift`. -/
abbrev LiftTransformer := TransformerLift → TransformerLift

/-- Splice a countable family of transformer lifts.

Output coordinate `i` selects lift `i`; the selected transformer uses its
canonical output coordinate `0`, exactly as `spliceTransformers` does one level
below.
-/
```

```
def spliceLifts (lifts : Nat → TransformerLift) : TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)

/-- Promote a transformer lift by one schedule-driven hyperreflection step.

This abstracts the successor clause of `diagonalLift`: the incoming lift is
used both to amplify the base transformer and as the local lifting operation
at every node of the schedule.
-/
def promoteLift (schedule : WTree Nat) : LiftTransformer :=
  fun lift base => stageLift lift (lift base) schedule

/-- Iterate a lift transformer along a well-founded tree.

This is the exact next-type analogue of `stageLift`.
-/
def liftStage (amp : LiftTransformer) (base : TransformerLift) :
    WTree Nat → TransformerLift
  | .leaf => amp base
  | .node f =>
      amp (spliceLifts (fun i => liftStage amp base (f i)))

/-- One autonomous third-order closure.

The same schedule is used internally by `promoteLift` and externally by
`liftStage`.  Thus the schedule controls both the local hyperreflection and the
branching reflection over whole lifts.
-/
def superClosure (schedule : WTree Nat) (base : TransformerLift) :
    TransformerLift :=
  liftStage (promoteLift schedule) base schedule

/-- Iterate the third-order closure through a finite meta-rank. -/
def superIterate (schedule : WTree Nat) :
    Nat → TransformerLift → TransformerLift
  | 0, base => base
  | k + 1, base => superClosure schedule (superIterate schedule k base)

end BusyLean
```

Listing 7: BusyLean/Contender20.lean

```
import BusyLean.SuperReflection
import BusyLean.Contender19

/-!
# Contender 20: third-order autonomous hyperreflection

The preceding construction already forms a hierarchy of transformer lifts.
This contender reflects over those lifts themselves.

At input `n`:

1. `family19 n = F19 (F19 n)` supplies the preceding construction as a seed;
2. `schedule20 n` is an autonomous W-tree with `family19 n + 42` higher
   closures above `tree12`;
3. an ordinary `diagonalLift` of rank `family19 n + 1` is the initial lift;
4. `superIterate` applies a third-order autonomous closure
   `family19 n + 1` times to that entire lift;
5. the resulting lift acts on large-Veblen reflection and yields a countable
```

```
   family;
6. one final large-Veblen reflection sees that whole family.

All recursion is structural and the definition is a closed computable `Nat`.
Never evaluate `contender_20`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Contender 19's construction as a function of its input. -/
def family19 (n : Nat) : Nat := F19 (F19 n)

/-- Contender 19's construction repeated at every seed coordinate. -/
def seed19 : Family := fun _ => family19

/-- A previous-number-controlled autonomous schedule. -/
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42) (Autonomous.encode tree12))

/-- Rank of the initial, current-style diagonal lift. -/
def innerRank20 (n : Nat) : Nat := family19 n + 1

/-- Rank of the new third-order iteration. -/
def metaRank20 (n : Nat) : Nat := family19 n + 1

/-- The third-order lift used at input `n`. -/
def lift20 (n : Nat) : TransformerLift :=
  superIterate (schedule20 n) (metaRank20 n)
    (diagonalLift (schedule20 n) (innerRank20 n))

/-- The complete family produced by the third-order lift. -/
def hyperFamily20 (n : Nat) : Family :=
  lift20 n lTransformer seed19

/-- One final large-Veblen reflection over the complete third-order family. -/
def F20 (n : Nat) : Nat :=
  reflectLFamily (hyperFamily20 n) n

/-- Contender 20.  Never evaluate it. -/
def contender_20 : Nat := F20 (F20 42)

end BusyLean
```

B Complete proof modules

Listing 8: BusyLean/SuperReflectionFacts.lean

```
import BusyLean.SuperReflection
import BusyLean.HyperReflectionFacts

/-!
# Growth facts for third-order autonomous hyperreflection

The crucial invariant is `GoodLift`: a transformer lift preserves progressive
```

```
transformers and is strictly inflationary on every progressive transformer.
The proofs below never assume semantic monotonicity in an oracle family.  Such
monotonicity is false for reflective object languages because programs may
branch on oracle values.  Instead every inequality follows an explicit visible
coordinate or the explicit zero child of a tree node.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- A lift has both properties needed by the higher construction. -/
def GoodLift (lift : TransformerLift) : Prop :=
  PreservesProgressive lift ∧ StrictlyInflationary lift

/-- A lift transformer maps good lifts to good lifts. -/
def PreservesGoodLifts (amp : LiftTransformer) : Prop :=
  ∀ lift, GoodLift lift → GoodLift (amp lift)

/-- A lift transformer strictly exceeds its input lift on every progressive
base transformer and every visible coordinate. -/
def LiftStrictlyInflationary (amp : LiftTransformer) : Prop :=
  ∀ lift, GoodLift lift →
    ∀ (base : Transformer), Progressive base →
    ∀ (seed : Family) (i o n : Nat), i ≤ n →
      lift base seed i n < amp lift base seed o n

/-- Coordinatewise splicing preserves the good-lift invariant. -/
theorem spliceLifts_good (lifts : Nat → TransformerLift)
    (hlifts : ∀ i, GoodLift (lifts i)) :
    GoodLift (spliceLifts lifts) := by
  constructor
  · intro base hbase
    exact spliceTransformers_progressive
      (fun i => (hlifts i).1 base hbase)
  · intro base hbase seed i o n hi
    change base seed i n < lifts o base seed 0 n
    exact (hlifts o).2 base hbase seed i 0 n hi

/-- Promotion maps every good lift to another good lift. -/
theorem promoteLift_good (schedule : WTree Nat) (lift : TransformerLift)
    (hlift : GoodLift lift) :
    GoodLift (promoteLift schedule lift) := by
  constructor
  · intro base hbase
    exact stageLift_progressive lift hlift.1
      (lift base) (hlift.1 base hbase) schedule
  · intro base hbase seed i o n hi
    have hfirst : base seed i n < lift base seed i n :=
      hlift.2 base hbase seed i i n hi
    have hsecond :
        lift base seed i n <
          stageLift lift (lift base) schedule seed o n :=
      base_lt_stageLift lift hlift.1 hlift.2
        (lift base) (hlift.1 base hbase)
        schedule seed i o n hi
    exact Nat.lt_trans hfirst hsecond

/-- Promotion preserves good lifts. -/
```

```
theorem promoteLift_preservesGood (schedule : WTree Nat) :
    PreservesGoodLifts (promoteLift schedule) :=
  fun lift hlift => promoteLift_good schedule lift hlift

/-- Promotion strictly exceeds the promoted lift itself. -/
theorem lift_lt_promoteLift (schedule : WTree Nat)
    (lift : TransformerLift) (hlift : GoodLift lift)
    (base : Transformer) (hbase : Progressive base)
    (seed : Family) (i o n : Nat) (hi : i ≤ n) :
    lift base seed i n < promoteLift schedule lift base seed o n :=
  base_lt_stageLift lift hlift.1 hlift.2
    (lift base) (hlift.1 base hbase)
    schedule seed i o n hi

/-- Promotion is strictly inflationary at the lift level. -/
theorem promoteLift_liftStrict (schedule : WTree Nat) :
    LiftStrictlyInflationary (promoteLift schedule) :=
  fun lift hlift base hbase seed i o n hi =>
    lift_lt_promoteLift schedule lift hlift base hbase seed i o n hi

/-- Tree iteration preserves the good-lift invariant. -/
theorem liftStage_good (amp : LiftTransformer)
    (hamp : PreservesGoodLifts amp)
    (baseLift : TransformerLift) (hbaseLift : GoodLift baseLift) :
    (a : WTree Nat) → GoodLift (liftStage amp baseLift a)
  | .leaf => hamp baseLift hbaseLift
  | .node f => by
      apply hamp
      exact spliceLifts_good
        (fun i => liftStage amp baseLift (f i))
        (fun i => liftStage_good amp hamp baseLift hbaseLift (f i))

/-- If the local lift transformer is strict, every tree stage strictly exceeds
its original base lift.

The node case follows child zero explicitly, identifies it with coordinate zero
of the splice, and then applies strictness at the parent. -/
theorem baseLift_lt_liftStage (amp : LiftTransformer)
    (hpres : PreservesGoodLifts amp)
    (hstrict : LiftStrictlyInflationary amp)
    (baseLift : TransformerLift) (hbaseLift : GoodLift baseLift) :
    (a : WTree Nat) →
      ∀ (base : Transformer), Progressive base →
      ∀ (seed : Family) (i o n : Nat), i ≤ n →
        baseLift base seed i n < liftStage amp baseLift a base seed o n
  | .leaf, base, hbase, seed, i, o, n, hi =>
      hstrict baseLift hbaseLift base hbase seed i o n hi
  | .node f, base, hbase, seed, i, o, n, hi => by
      have hchild :
          baseLift base seed i n <
            liftStage amp baseLift (f 0) base seed 0 n :=
        baseLift_lt_liftStage amp hpres hstrict baseLift hbaseLift
          (f 0) base hbase seed i 0 n hi
      have hchildren :
          ∀ j, GoodLift (liftStage amp baseLift (f j)) :=
        fun j => liftStage_good amp hpres baseLift hbaseLift (f j)
      have hsplice :
          GoodLift (spliceLifts (fun j => liftStage amp baseLift (f j))) :=
        spliceLifts_good
          (fun j => liftStage amp baseLift (f j)) hchildren
```

```
      have hparent :
          liftStage amp baseLift (f 0) base seed 0 n <
            liftStage amp baseLift (.node f) base seed o n := by
        change
          spliceLifts (fun j => liftStage amp baseLift (f j))
              base seed 0 n <
            amp (spliceLifts (fun j => liftStage amp baseLift (f j)))
              base seed o n
        exact hstrict
          (spliceLifts (fun j => liftStage amp baseLift (f j)))
          hsplice base hbase seed 0 o n (Nat.zero_le n)
      exact Nat.lt_trans hchild hparent

/-- One autonomous third-order closure preserves the good-lift invariant. -/
theorem superClosure_good (schedule : WTree Nat)
    (lift : TransformerLift) (hlift : GoodLift lift) :
    GoodLift (superClosure schedule lift) :=
  liftStage_good (promoteLift schedule)
    (promoteLift_preservesGood schedule) lift hlift schedule

/-- One autonomous third-order closure strictly exceeds its input lift. -/
theorem lift_lt_superClosure (schedule : WTree Nat)
    (lift : TransformerLift) (hlift : GoodLift lift)
    (base : Transformer) (hbase : Progressive base)
    (seed : Family) (i o n : Nat) (hi : i ≤ n) :
    lift base seed i n < superClosure schedule lift base seed o n :=
  baseLift_lt_liftStage (promoteLift schedule)
    (promoteLift_preservesGood schedule)
    (promoteLift_liftStrict schedule)
    lift hlift schedule base hbase seed i o n hi

/-- Every finite meta-rank is a good lift. -/
theorem superIterate_good (schedule : WTree Nat)
    (initial : TransformerLift) (hinitial : GoodLift initial) :
    ∀ k, GoodLift (superIterate schedule k initial)
  | 0 => hinitial
  | k + 1 =>
      superClosure_good schedule
        (superIterate schedule k initial)
        (superIterate_good schedule initial hinitial k)

/-- Convenient progressivity projection for a finite meta-rank. -/
theorem superIterate_preservesProgressive (schedule : WTree Nat)
    (initial : TransformerLift) (hinitial : GoodLift initial) (k : Nat) :
    PreservesProgressive (superIterate schedule k initial) :=
  (superIterate_good schedule initial hinitial k).1

/-- Convenient strictness projection for a finite meta-rank. -/
theorem superIterate_strictlyInflationary (schedule : WTree Nat)
    (initial : TransformerLift) (hinitial : GoodLift initial) (k : Nat) :
    StrictlyInflationary (superIterate schedule k initial) :=
  (superIterate_good schedule initial hinitial k).2

/-- The meta-rank hierarchy is strictly increasing at every successor. -/
theorem superIterate_step_lt (schedule : WTree Nat) (k : Nat)
    (initial : TransformerLift) (hinitial : GoodLift initial)
    (base : Transformer) (hbase : Progressive base)
    (seed : Family) (i o n : Nat) (hi : i ≤ n) :
    superIterate schedule k initial base seed i n <
      superIterate schedule (k + 1) initial base seed o n := by
```

```
  simpa [superIterate] using
    (lift_lt_superClosure schedule
      (superIterate schedule k initial)
      (superIterate_good schedule initial hinitial k)
      base hbase seed i o n hi)

/-- Every positive meta-rank strictly exceeds the initial lift. -/
theorem lift_lt_superIterate_succ (schedule : WTree Nat)
    (initial : TransformerLift) (hinitial : GoodLift initial) :
    ∀ (k : Nat) (base : Transformer), Progressive base →
      ∀ (seed : Family) (i o n : Nat), i ≤ n →
        initial base seed i n <
          superIterate schedule (k + 1) initial base seed o n
  | 0, base, hbase, seed, i, o, n, hi => by
      simpa using
        (superIterate_step_lt schedule 0 initial hinitial
          base hbase seed i o n hi)
  | k + 1, base, hbase, seed, i, o, n, hi => by
      have hfirst :
          initial base seed i n <
            superIterate schedule (k + 1) initial base seed i n :=
        lift_lt_superIterate_succ schedule initial hinitial
          k base hbase seed i i n hi
      have hsecond :
          superIterate schedule (k + 1) initial base seed i n <
            superIterate schedule ((k + 1) + 1) initial base seed o n :=
        superIterate_step_lt schedule (k + 1) initial hinitial
          base hbase seed i o n hi
      exact Nat.lt_trans hfirst hsecond

end BusyLean
```

Listing 9: BusyLean/Comparison20.lean

```
import BusyLean.Contender20
import BusyLean.SuperReflectionFacts
import BusyLean.Comparison19

/-! # Contender 20: comparison theorems -/

namespace BusyLean

open BusyLean.W

/-- The current-style lift used as the base of Contender 20 satisfies both
higher invariants. -/
theorem diagonalLift20_good (n : Nat) :
    GoodLift (diagonalLift (schedule20 n) (innerRank20 n)) :=
  ⟨diagonalLift_preservesProgressive
      (schedule20 n) (innerRank20 n),
    diagonalLift_strictlyInflationary
      (schedule20 n) (innerRank20 n)⟩

/-- The final third-order lift is good. -/
theorem lift20_good (n : Nat) : GoodLift (lift20 n) := by
  simpa [lift20] using
    (superIterate_good (schedule20 n)
      (diagonalLift (schedule20 n) (innerRank20 n))
      (diagonalLift20_good n) (metaRank20 n))
```

```
/-- `lift20 n` preserves progressive transformers. -/
theorem lift20_preservesProgressive (n : Nat) :
    PreservesProgressive (lift20 n) :=
  (lift20_good n).1

/-- `lift20 n` strictly exceeds every progressive base transformer. -/
theorem lift20_strictlyInflationary (n : Nat) :
    StrictlyInflationary (lift20 n) :=
  (lift20_good n).2

/-- Formal architectural gain: the complete third-order lift strictly exceeds
its own initial current-style diagonal hyperlift, at every visible coordinate.
-/
theorem diagonalLift_lt_lift20 (n : Nat)
    (base : Transformer) (hbase : Progressive base)
    (seed : Family) (i o : Nat) (hi : i ≤ n) :
    diagonalLift (schedule20 n) (innerRank20 n)
        base seed i n <
      lift20 n base seed o n := by
  simpa [lift20, metaRank20] using
    (lift_lt_superIterate_succ (schedule20 n)
      (diagonalLift (schedule20 n) (innerRank20 n))
      (diagonalLift20_good n) (family19 n)
      base hbase seed i o n hi)

/-- The preceding family is already beaten inside the produced hyper-family. -/
theorem family19_lt_hyperFamily20 (n : Nat) :
    family19 n < hyperFamily20 n 0 n := by
  have h := (lift20_preservesProgressive n)
    lTransformer lTransformer_progressive seed19 0 0 n (Nat.zero_le n)
  simpa [seed19, hyperFamily20] using h

/-- Every visible coordinate of the produced family is below the final
large-Veblen reflection. -/
theorem hyperFamily20_lt_F20 {i n : Nat} (hi : i ≤ n) :
    hyperFamily20 n i n < F20 n := by
  simpa [F20] using
    (oracle_lt_reflectLFamily (hyperFamily20 n) (i := i) (n := n) hi)

/-- At every input, `F20` exceeds Contender 19's complete family. -/
theorem family19_lt_F20 (n : Nat) : family19 n < F20 n :=
  Nat.lt_trans (family19_lt_hyperFamily20 n)
    (hyperFamily20_lt_F20 (n := n) (i := 0) (Nat.zero_le n))

/-- Even the raw large-Veblen reflection over the old seed is below the
third-order family. -/
theorem raw_reflection_lt_hyperFamily20 (n : Nat) :
    reflectLFamily seed19 n < hyperFamily20 n 0 n := by
  have h := (lift20_strictlyInflationary n)
    lTransformer lTransformer_progressive seed19 n 0 n (Nat.le_refl n)
  simpa [lTransformer, hyperFamily20] using h

/-- The raw large-Veblen reflection over the old seed is below `F20`. -/
theorem raw_reflection_lt_F20 (n : Nat) :
    reflectLFamily seed19 n < F20 n :=
  Nat.lt_trans (raw_reflection_lt_hyperFamily20 n)
    (hyperFamily20_lt_F20 (n := n) (i := 0) (Nat.zero_le n))

/-- Every enumerated large-Veblen-ladder program run against the final packed
third-order family is below `F20 n`. -/
```

```
theorem final_lvalue_lt_F20 {n : Nat}
    {t : FL.CTm Ordinal.LargeVeblen}
    (ht : t ∈ FL.lveblenLadderUpTo n) :
    FL.lvalue (packFamily (hyperFamily20 n)) n t < F20 n := by
  simpa [F20] using
    (lvalue_lt_reflectLFamily (hyperFamily20 n) ht)

/-- The numerical input is below the preceding family. -/
theorem input_lt_family19 (n : Nat) : n < family19 n := by
  simpa [family19] using
    (Nat.lt_trans (input_lt_F19 n) (input_lt_F19 (F19 n)))

/-- The numerical input is below `F20`. -/
theorem input_lt_F20 (n : Nat) : n < F20 n :=
  Nat.lt_trans (input_lt_family19 n) (family19_lt_F20 n)

/-- The required comparison: Contender 19 is strictly below Contender 20. -/
theorem contender_19_lt_contender_20 : contender_19 < contender_20 := by
  simpa [family19, contender_19, contender_20] using
    (Nat.lt_trans (family19_lt_F20 42)
      (input_lt_F20 (F20 42)))

/-- A convenient transitive comparison with Contender 18. -/
theorem contender_18_lt_contender_20 : contender_18 < contender_20 :=
  Nat.lt_trans contender_18_lt_contender_19 contender_19_lt_contender_20

end BusyLean
```

C Focused kernel audit

Listing 10: `test/Audit20.lean`

```
import BusyLean.Comparison20
import Lean

/-!
Focused kernel audit for Contender 20.

Run after installation with:

  lake env lean test/Audit20.lean
-/

open Lean Elab Command

/-- All constants reachable from `root` through types and values. -/
def constClosure20 (env : Environment) (root : Name) : NameSet := Id.run do
  let mut seen : NameSet := {}
  let mut stack := #[root]
  while stack.size > 0 do
    let c := stack.back!
    stack := stack.pop
    if seen.contains c then continue
    seen := seen.insert c
    let some ci := env.find? c | continue
    let body : Array Name := match ci with
      | .defnInfo v => v.value.getUsedConstants
      | .thmInfo v => v.value.getUsedConstants
```

```
      | .opaqueInfo v => v.value.getUsedConstants
      | _ => #[]
    stack := stack ++ ci.type.getUsedConstants ++ body
  return seen

/-- Fail if any listed constant is in the transitive closure of `root`. -/
elab "#assert_not_in_closure20 " root:ident " : " bad:ident* : command => do
  let env ← getEnv
  let r ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo root
  let cl := constClosure20 env r
  for b in bad do
    let n ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo b
    if cl.contains n then
      throwError m!"{r} depends on forbidden constant {n}"
  logInfo m!"{r}: closure of {cl.size} constants avoids all {bad.size} forbidden names"

#print axioms BusyLean.spliceLifts_good
#print axioms BusyLean.promoteLift_good
#print axioms BusyLean.baseLift_lt_liftStage
#print axioms BusyLean.superIterate_good
#print axioms BusyLean.lift_lt_superIterate_succ
#print axioms BusyLean.contender_20
#print axioms BusyLean.diagonalLift_lt_lift20
#print axioms BusyLean.family19_lt_F20
#print axioms BusyLean.final_lvalue_lt_F20
#print axioms BusyLean.contender_19_lt_contender_20

#assert_not_in_closure20 BusyLean.contender_20 :
  BusyLean.contender_19 BusyLean.contender_18 BusyLean.contender_17
  BusyLean.contender_16 BusyLean.contender_15 BusyLean.contender_14
  BusyLean.contender_13 BusyLean.contender_12 BusyLean.contender_11
  BusyLean.contender_10 BusyLean.contender_9 BusyLean.contender_8
  BusyLean.contender_7 BusyLean.contender_6 BusyLean.contender_5

#check @BusyLean.contender_19_lt_contender_20
#check @BusyLean.diagonalLift_lt_lift20
#check @BusyLean.final_lvalue_lt_F20
```

D Acceptance checklist

☐ Apply the overlay to the audited repository commit or review the patch against the intended newer commit.

☐ Run `python3scripts/check20.py` under Lean 4.34.0.

☐ Confirm the definition phase is at most 15 seconds.

☐ Confirm the proof phase is at most 60 seconds.

☐ Confirm every printed axiom set is a subset of the two permitted axioms `propext` and `Quot.sound`.

☐ Confirm the transitive closure of `contender_20` avoids every earlier closed contender constant.

☐ If any elaboration repair is required, record it as a separate minimal commit and rerun the complete audit.

☐ Integrate the four imports into `BusyLean.lean`, update the repository README and article, and add Contender 20 to the global audit only after all preceding checks pass.

References

[1] Scott Aaronson. *Who Can Name the Bigger Number?* 1999. Available at https://www.scottaaronson.com/writings/bignumbers.html.

[2] Cody Roux and contributors. *name-the-biggest-number*. GitHub repository, https://github.com/codyroux/name-the-biggest-number.

[3] Vladimir Reshetnikov and BusyLean contributors. *BusyLean: name the biggest number, in Lean*. GitHub repository, https://github.com/VladimirReshetnikov/BusyLean.

[4] Solomon Feferman. Transfinite recursive progressions of axiomatic theories. *Journal of Symbolic Logic*, 27(3):259–316, 1962.

[5] Kurt Schütte. *Proof Theory*. Springer, 1977.

[6] Wilfried Buchholz. A new system of proof-theoretic ordinal functions. *Annals of Pure and Applied Logic*, 32:195–207, 1986.

[7] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635. Springer, 2021.